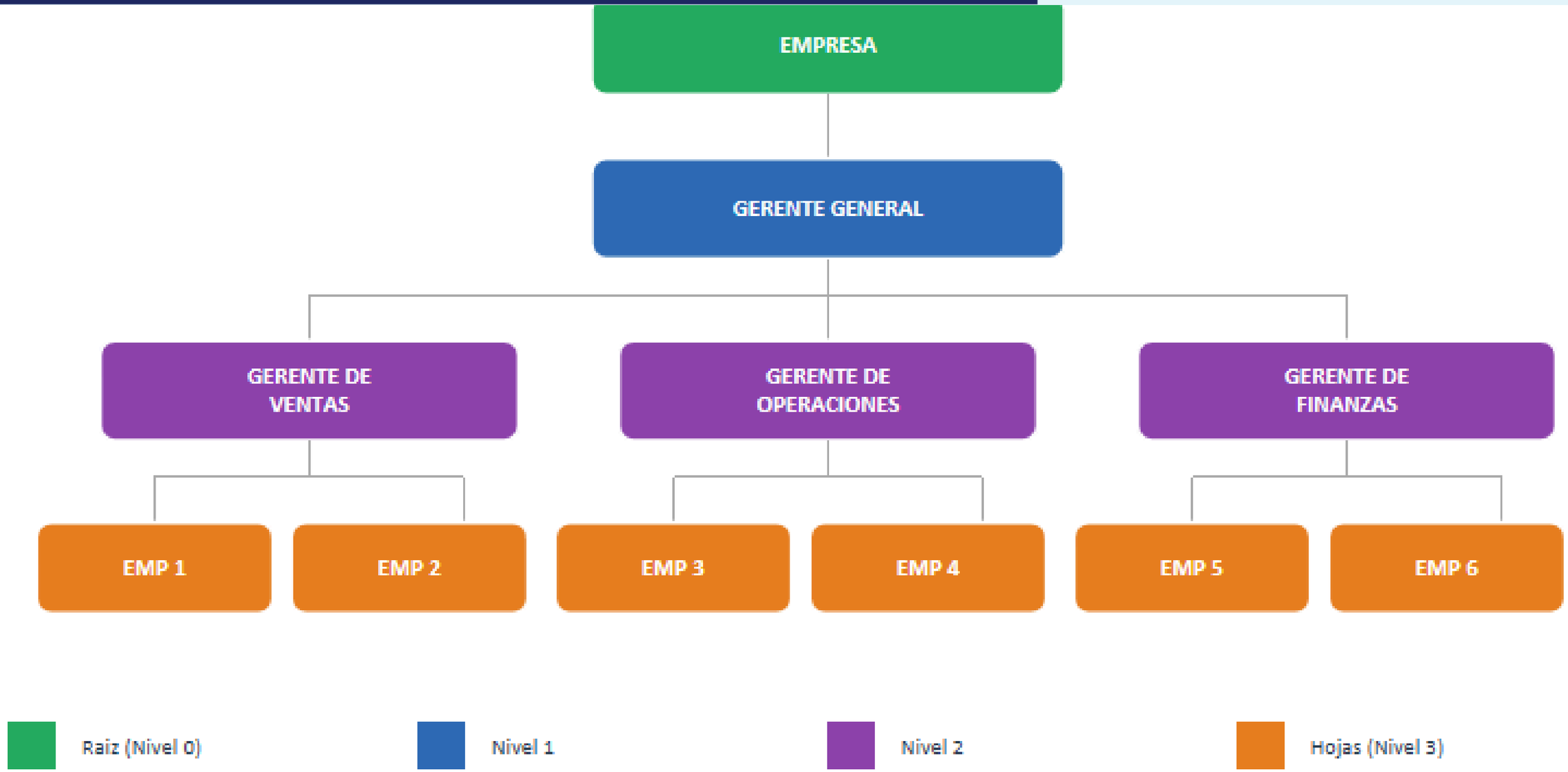
EJEMPLO

Raiz: nodo inicial, sin padre.
Nodo intermedio: tiene hijos.
Hoja: nodo sin hijos.

Raiz	Nodo inicial del arbol (EMPRESA)
Nivel 1	Gerente General
Nivel 2	Gerentes de area
Hojas	Empleados (sin hijos)

Estructura del Arbol - Empresa



Estructura del Nodo en C++

```cpp
// Estructura de un nodo del arbol
struct Nodo {
    string cargo;   // nombre del cargo
    Nodo* izq;    // hijo izquierdo
    Nodo* centro; // hijo del medio
    Nodo* der;    // hijo derecho

    // Constructor del nodo
    Nodo(string c) {
        cargo  = c;
        izq    = NULL;
        centro = NULL;
        der    = NULL;
    }
};
```

Campos del struct:

cargo: Guarda el nombre del empleado o gerente

izq: Puntero al hijo izquierdo

centro: Puntero al hijo del centro

der: Puntero al hijo derecho

NULL: Sin hijo = puntero nulo

Construyendo el Arbol en main()

```
// Nivel 0 - Raiz
Nodo* empresa = new Nodo("EMPRESA");

// Nivel 1 - Gerente General
empresa->izq = new Nodo("GERENTE GENERAL");
Nodo* gg = empresa->izq;

// Nivel 2 - Tres gerentes
gg->izq    = new Nodo("GERENTE DE VENTAS");
gg->centro = new Nodo("GERENTE DE OPERACIONES");
gg->der    = new Nodo("GERENTE DE FINANZAS");

// Nivel 3 - Empleados (hojas) bajo ventas
gg->izq->izq = new Nodo("EMPLEADO 1");  gg->izq->der = new Nodo("EMPLEADO 2");
```

Funcion mostrarArbol() - Recursion

```
void mostrarArbol(Nodo* raiz, int nivel) {
    if (raiz == NULL) return;

    // Imprimir sangria segun nivel
    for (int i = 0; i < nivel; i++)
        cout << "    ";
    cout << "|-- " << raiz->cargo << endl;

    // Llamadas recursivas a hijos
    mostrarArbol(raiz->izq,    nivel + 1);
    mostrarArbol(raiz->centro, nivel + 1);
    mostrarArbol(raiz->der,    nivel + 1);
}
```

Como funciona la recursion:

1. Caso base: si el nodo es NULL, termina.
2. Imprime espacios segun el nivel de profundidad.
3. Muestra el cargo del nodo actual.
4. Llama a si misma con cada hijo (izq, centro, der).
5. Incrementa el nivel +1 en cada llamada.

Salida en Consola

output - C++

```
=== ESTRUCTURA ORGANIZACIONAL ===

|-- EMPRESA
    |-- GERENTE GENERAL
        |-- GERENTE DE VENTAS
            |-- EMPLEADO 1
            |-- EMPLEADO 2
        |-- GERENTE DE OPERACIONES
            |-- EMPLEADO 3
            |-- EMPLEADO 4
        |-- GERENTE DE FINANZAS
            |-- EMPLEADO 5
            |-- EMPLEADO 6
```

Analisis del recorrido:

- EMPRESA es la raiz (nivel 0), sin sangria.
- Gerente General en nivel 1 (4 espacios).
- Gerentes en nivel 2 (8 espacios).
- Empleados en nivel 3 (12 espacios), son hojas.
- Recorrido PRE-ORDEN: padre antes que hijos.

Liberacion de Memoria

```
// Liberar memoria post-orden
void liberarArbol(Nodo* raiz) {
    if (raiz == NULL) return;

    // Primero liberamos los hijos
    liberarArbol(raiz->izq);
    liberarArbol(raiz->centro);
    liberarArbol(raiz->der);

    // Despues liberamos el padre
    delete raiz;
}
```

! Importante: Siempre liberar la memoria que pedimos con new.

Orden POST-ORDEN:

1. Se libera primero el hijo izquierdo.
2. Luego el hijo central.
3. Luego el hijo derecho.
4. Finalmente el nodo padre (delete).